

OnRe Solana

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	RWA	Documentation quality	Medium 
Timeline	2025-05-15 through 2025-05-19	Test quality	Medium 
Language	Rust	Total Findings	5  Fixed: 3 Acknowledged: 2
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings 	1  Fixed: 1
Specification	None	Medium severity findings 	1  Fixed: 1
Source Code	onre-finance/onre-sol  #6aba335 	Low severity findings 	3  Fixed: 1 Acknowledged: 2
Auditors	- Paul Clemson Auditing Engineer - Darren Jensen Auditing Engineer - István Böhm Auditing Engineer	Undetermined severity 	0

ty findin gs	
Infor matio nal ⓘ findin gs	0

Summary of Findings

The Onre protocol is a Solana program built with the Anchor framework which facilitates the sale of real world assets via an offers system. The protocol allows an authorized user to create offers specifying the buy and sell tokens, a maximum amount of tokens available for sale, and a sale price that increases at intervals between the start and end of the offer period. The main focus of this audit looked at the addition of the dynamic pricing system. Overall, the codebase is well engineered and adheres to good security practices. The security of the project would be further improved by strengthening the codebase's test suite, and this audit review recommends it is done before the project is deployed. The audit review found some issues around the protocol's price calculation methods as well as a few other issues and it is recommended that the team fix each of them.

ID	DESCRIPTION	SEVERITY	STATUS
ONRE-1	Improper Price Calculation Leads to Overpayment	• High ⓘ	Fixed
ONRE-2	Missing <code>mut</code> on <code>boss</code> Signer Account in <code>CloseOfferTwo</code>	• Medium ⓘ	Fixed
ONRE-3	Lack of Slippage Protection Exposes Users to Unfavorable Execution Prices	• Low ⓘ	Acknowledged
ONRE-4	Improper Values in the <code>OfferTakenOne</code> and <code>OfferTakenTwo</code> Event Emission	• Low ⓘ	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
ONRE-5	Initialization Can Be Frontrun	• Low ⓘ	Acknowledged

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.



Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.

2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Operational Considerations

- The boss role in the protocol is trusted to ensure the correct tokens are offered at the correct prices and that the price intervals progress in a reasonable manner.
- The boss will be responsible for opening redemption windows for users, and performing any necessary KYC. However this is outside the scope of our assessment.
- The Associated Token Accounts (ATAs) used within the protocol are expected to have been created prior to interacting with the program.
- The team must ensure that tokens used in the protocol have a reasonable number of decimals to prevent overflows or cause significant rounding errors.
- A multi-signature wallet should be used for sensitive roles, and all keys and secrets related to the smart contract are secured properly.

Key Actors And Their Capabilities

The following roles are present in the protocol:

Boss:

- Can transfer the boss role to another account.
- Can create new token offers with one or two buy tokens.
- Can fund the created offers with buy tokens.
- Can close offers to reclaim the collected sell tokens and any remaining buy tokens.

User:

- Users can accept an active offer by providing the required sell tokens in exchange for the buy tokens.

Findings

ONRE-1

Improper Price Calculation Leads to Overpayment

• High ⓘ

Fixed

Marked as "Fixed" by the client.

Addressed in: `8a41021c9694d5735ef2cd692f6d1f26bcae2c3a` .

Additional fix in: `3782a7aff178e86adc427b3d8e151efc4076db3e` .

File(s) affected: `take_offer.rs`

Description: When an offer's total duration is not evenly divisible `price_fix_duration` , a partial interval will occur at the end of the offer period that is not accounted for in the protocol's pricing calculation. The price calculation logic fails to properly handle this edge case, causing users who transact during this period to pay one interval more than the intended maximum price (`sell_token_end_amount`).

An additional note here is that in the current implementation this behaviour will also occur if a user calls one of the take offer functions in the final second of the offer, because of the inclusive upper bounds check on `offer.offer_end_time` .

Consider this simplified example:

1. The team creates an offer where the duration from start to end is 2.5 days.
2. The `price_fix_duration` is set to one day.
3. Assuming the start price is 10e9 and the end price is 20e9.
4. The `number_of_intervals` used in `calculate_current_sell_amount()` will round down to two one day long intervals due to integer division (`216_000 // 86_400`).
5. Users transacting on day one will pay the start price plus one interval price (15e9).
6. Users transacting on day two will pay the start price plus two interval prices, this will be the intended `sell_token_end_amount` ratio (20e9).
7. However users transacting in the additional half day will be charged the start price plus 3 interval bumps, causing them to pay a greater amount (25e9) than the intended `sell_token_end_amount` ratio.

Recommendation: Consider either updating the price calculate to include fractional intervals, or ensure that fractional intervals cannot exist.

ONRE-2

Missing `mut` on `boss` Signer Account in

• Medium ⓘ

Fixed

`CloseOfferTwo`

✓ Update

Marked as "Fixed" by the client.

Addressed in: `a62ac51db21131bbab838178877527ac977cd985` .

File(s) affected: `programs/onreapp/src/instructions/close_offer.rs`

Description: The `pub boss: Signer<'info> account in CloseOfferTwo` is not marked as mutable. The boss account is intended to receive rent refunds from closing the offer account and associated token accounts, requiring it to be mutable (`#[account(mut)]`). This missing constraint may cause the `close_offer_two()` instruction to fail.

Recommendation: Add the `#[account(mut)]` constraint to the `boss` field in the `CloseOfferTwo` struct.

ONRE-3

Lack of Slippage Protection Exposes Users to Unfavorable Execution Prices

• Low ⓘ

Acknowledged

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

We are aware of this risk. For now, we are leaving it as is.

File(s) affected: `programs/onreapp/src/instructions/take_offer.rs`

Description: The `take_offer::take_offer_*` instruction enables users to purchase `buy_tokens` by providing a `sell_token_amount`. The exchange rate is determined by the offer's `current_interval` at execution time.

However, users who submit transactions near interval boundaries face a risk: if the interval changes before transaction execution, users may receive fewer `buy_tokens` than expected due to the change in price, with no mechanism to specify acceptable limits.

Recommendation: Implement slippage protection by adding a `min_buy_tokens` parameter to the `take_offer` instructions. This would allow users to specify the minimum acceptable quantity of `buy_tokens` they expect to receive for their `sell_token_amount`, protecting them from unfavorable price movements.

ONRE-4

Improper Values in the `OfferTakenOne` and `OfferTakenTwo` Event Emission

• Low ⓘ

Fixed

Update

Marked as "Fixed" by the client.

Addressed in: `1bb8b7da94201493bca1524f9ebdea3be60ce45f`.

File(s) affected: `programs/onreapp/src/instructions/take_offer.rs`

Description: The `OfferTakenOne` and `OfferTakenTwo` events include a `remaining_sell_token_amount` field. This is calculated (`sell_token_end_amount - offer_sell_token_account.amount`) instead of the remaining buy token amounts. This value may be incorrect as the am.

Recommendation: Consider either editing the `remaining_sell_token_amount` calculation or removing it from the event.

ONRE-5 Initialization Can Be Frontrun

• Low ⓘ

Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

We are aware of this possibility. When the contract is deployed, it will be initialized right away through Squads. There is a check added and the transaction fails, if the boss is already set.

File(s) affected: `programs/onreapp/src/instructions/initialize.rs`

Description: The `initialize()` function can be frontrun after the program is deployed. An attacker might create a state and set themselves as a `boss`, preventing the real admin from controlling the program.

Recommendation: Consider enforcing a known hard-coded `boss` key during initialization or use an alternative method to prevent front-running attacks.

Auditor Suggestions

S1 Unnecessary `mut` on Signer Accounts

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `eaafa09fd44c96067cf31a8c24178fe0e9ffe3870` .

File(s) affected: `programs/onreapp/src/instructions/set_boss.rs`,
`programs/onreapp/src/instructions/take_offer.rs`

Description: - The `pub boss: Signer<'info>` account in `SetBoss` is marked as mutable (`#[account(mut)]`).

- The `pub user: Signer<'info>` accounts in `TakeOfferOne` and `TakeOfferTwo` are marked as mutable (`#[account(mut)]`).

These instructions only read the `boss` and `user` accounts' keys and do not modify their data or lamports. Accounts do not need to be mutable unless their lamports are being directly debited or credited by the program logic or their data is being changed.

Recommendation: Consider removing the `#[account(mut)]` constraint from the `boss` and `user` accounts to adhere to the principle of least privilege and improve code clarity.

S2

Critical Role Transfer Not Following Two-Step Pattern

Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

Will be added in the future. The boss account is not going to be transferred frequently and it is a Squads account.

File(s) affected: `programs/onreapp/src/instructions/set_boss.rs`

Description: The owner of the contracts can call `set_boss()` to transfer the ownership to a new address. If an uncontrollable address is accidentally provided as the new boss address then the contract will no longer have an active boss.

Recommendation: Consider allowing the transfer to happen over two steps, in which the following happens: 1 - The first call sets a `pending_boss` value in storage. 2 - This `pending_boss` can then call a second function to accept the role transfer.

This would ensure that the role is always transferred to an address capable of fulfilling the role.

S3

Missing Validation of Buy Tokens in `close_offer_two()`

Fixed

Update

Marked as "Fixed" by the client.

Addressed in: 440f10a2c92c3fd4c800c5ca650219f2af3773bb .

File(s) affected: programs/onreapp/src/instructions/close_offer.rs

Description: The `close_offer_two()` function lacks an explicit check to ensure it's called on an offer with two buy tokens, unlike `close_offer_one()` which explicitly checks that the offer in question only has a single buy token.

Recommendation: Consider adding the following check to the `close_offer_two()` function:

```
require!(  
    ctx.accounts.offer.buy_token_2.mint != Pubkey::default(),  
    CloseOfferErrorCode::InvalidCloseOffer  
);
```

S4

Code Duplication Increases Maintenance Overhead and Error Risk

Mitigated

Update

Marked as "Mitigated" by the client.

Addressed in: efdeb438f4d0bdb7f3ceba59fa663c4af54b65fa .

The client provided the following explanation:

The validation logic was extracted into helper functions, to eliminate the duplication between `make_offer_one` and `make_offer_two`.

Additional change in : 5a38fb1c7e3872c49591932e2130eda4864a6ea0 .

File(s) affected: programs/onreapp/src/instructions/make_offer.rs

Description: Significant code duplication exists between the `make_offer_one` and `make_offer_two` functions, as well as in the `MakeOfferOne` and `MakeOfferTwo` structs. This redundancy creates technical debt and increases the risk of inconsistencies when modifying the program in the future.

Recommendation: Extract common validation logic into helper functions to eliminate duplication between `make_offer_one` and `make_offer_two` . Additionally, enhance the existing

`MakeOfferContext` trait to serve as a base for common account structure logic, potentially implementing an `initialize_offer_common` function to centralize shared initialization code.

S5

Missing Minimum Offer Duration Validation May Prevent Users From Responding to Offers

Acknowledged

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

The offers will be created by boss accounts, who will be responsible for making the offer with a reasonable duration.

File(s) affected: `programs/onreapp/src/instructions/make_offer.rs`

Description: The `make_offer::make_offer_*` functions allow bosses to create new offers without validating against a minimum offer duration. This oversight permits the creation of offers with extremely short durations (as brief as 1 second), potentially preventing users from having reasonable opportunities to respond.

Recommendation: Implement validation checks for offer durations to ensure they fall within practical timeframes, enhancing fairness and usability of the platform. Consider establishing a minimum duration parameter that provides adequate time for user interaction while maintaining operational efficiency.

S6 Code Improvements

Mitigated

Update

Marked as "Mitigated" by the client.

Addressed in: `e7e3f918eae976136fc6baa475bd51caa3acb67c`.

The client provided the following explanation:

Developer comments in `state.rs` were fixed to match the field names defined in the struct, unused `InvalidCurrentTime` error code was removed, an additional error code was introduced to distinguish between which insufficient balance errors when working with two buy tokens.

File(s) affected: `programs/onreapp/src/instructions/state.rs`,
`programs/onreapp/src/instructions/make_offer.rs`,

```
programs/onreapp/src/instructions/take_offer.rs ,  
programs/onreapp/src/instructions/close_offer.rs
```

Description: - In `state.rs` the developer comments for the `Offer` struct do not match the field names defined in the struct.

- Throughout the protocol code there is the comment *"This account is marked with CHECK as it's validated by the seed derivation."* applied to `AccountInfo` definitions. However the implementation is missing for the actual `CHECK` directive above the account attribute that Anchor requires to bypass validation. Consider adding the `CHECK` directive in the following format `/// CHECK: explanation` above the `AccountInfo` attribute.
- In `make_offer.rs` the `MakeOfferErrorCode` includes the definition for `InvalidCurrentTime` error code which is unused and can be removed.
- In `take_offer.rs` the `take_offer_two()` function will return the same error code `TakeOfferErrorCode::InsufficientOfferBalance` if buy token 1 or buy token 2 has insufficient balance making it challenging for the client to determine which token caused the failure. Consider using two separate error codes - one for each token.

Recommendation: Consider updating the code with improvements suggested.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Toolset

The notes below outline the setup and steps performed in the process of this audit.

Setup

Tool Setup:

- [Cargo Audit](#) [🔗](#) 0.21.2

Steps taken to run the tools:

- Installed via `cargo install cargo-audit`
- Ran `cargo audit`

Automated Analysis

Cargo Audit

Cargo audit found one potential issue in one of the project's crate dependencies. It is recommended that the team look into this and ensure it does not affect their codebase negatively.

```
Crate:      borsh
Version:    0.10.3
Warning:    unsound
Title:      Parsing borsh messages with ZST which are not-copy/clone is
            unsound
Date:       2023-04-12
ID:         RUSTSEC-2023-0033
URL:        https://rustsec.org/advisories/RUSTSEC-2023-0033
```

Test Suite Results

The following steps are required to run the test suite:

```
anchor build
npm install
anchor test
```

Which produces the following output:

```
Test Suites: 3 passed, 3 total
Tests:       45 passed, 45 total
```

Snapshots: 0 total
Time: 36.005 s

Code Coverage

The test coverage of the codebase has improved since the initial audit of the codebase, and now has significantly more robust testing of the programs.

Changelog

- 2025-05-19 - Initial report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source

or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

